

Sol Current-State Audit

Remaining Work, Risks, and the Shortest End-to-End Path

Repository state at commit 4dcadde - August 25, 2026

1 Executive Assessment

Sol currently has a credible end-to-end **executable language model**:

```
source/package
-> lexer and parser
-> syntax validation
-> HIR/name resolution
-> type/effect/contract analysis
-> owning typed IR
-> ownership validation
-> reference interpreter
-> sol test
```

This is real functionality rather than scaffolding. Records, enums, tuples, bounded generics, traits, effects, capabilities, contracts-as-obligations, ownership, mutation, loops, handlers, recursive patterns, cleanup, and host operations cross the complete pipeline.

Sol is not yet an end-to-end **application toolchain**. There is no `sol run`, application entrypoint model, standard host capability profile, runtime contract enforcement, MIR, native or WebAssembly backend, linker, runtime ABI, `sol build`, package manifest, dependency system, host wiring, or public semantic IR.

Current verification is healthy:

- Normal test suite: 35/35 passed.
- Clean ASan/UBSan suite: 35/35 passed.
- Worktree was clean except the intentionally untracked session file.

2 Current State

2.1 Source, Parsing, and Packages

The compiler has deterministic single-file and directory-package loading, a lossless lexer, a recovering arena parser, exhaustive syntax-tree validation, canonical formatting, multi-file modules, imports, and stable top-level semantic IDs.

The package model remains intentionally bounded. It has no manifests, dependency graph, lockfile, feature policy, aliases, re-exports, package-qualified identity, or host configuration. Directory packages are aggregated into package-global syntax and semantic arenas.

2.2 Semantic Pipeline

Name resolution, bounded first-order generics, records, enums, tuples, structural function types, traits, method evidence, effects, capabilities, exact handlers, contracts, recursive patterns, and exhaustiveness are operational.

The current HIR is primarily syntax-indexed semantic metadata rather than an independent typed semantic representation. Types, effects, contracts, and resolutions are maintained in parallel tables indexed by syntax IDs. This is effective for the bootstrap but will complicate incremental compilation and public semantic APIs.

2.3 Ownership and IR

The compiler lowers successful programs to a genuine self-contained owning typed IR. The IR retains executable types, calls, places, effects, contract templates, cleanup, pattern trees, and dispatch metadata. Ownership

checking covers affine moves, structural Copy, loans, partial moves, definite initialization, projected mutation, inout writeback, loops, regions, and deterministic cleanup.

This IR is an unstable interpreter IR. It is not a control-flow MIR, stable serialized Sol IR, target-independent layout IR, or safe hostile-input format.

2.4 Interpreter and CLI

The deterministic reference interpreter executes the bounded core with checked arithmetic, aggregate values, functions, callbacks, traits, capabilities, exact handlers, mutation, loops, patterns, cleanup, host operations, and explicit resource limits.

The implemented commands are `sol check`, `sol test`, `sol effects`, `sol inspect`, and `sol fmt`. The only user-facing execution path is authored Boolean tests. There is no application entrypoint, `sol run`, `sol build`, or executable artifact.

2.5 Contracts and Verification

Contracts, refined predicates, loop invariants, decreases measures, snapshots, and obligations are parsed, typed, purity-checked, and retained as deterministic templates. They are not enforced or discharged. The interpreter's contract-check policy exists at the API level but is unsupported.

This is the largest semantic gap relative to Sol's central claim: accepted contract syntax currently means that an obligation is well-formed, not that it has been proved or checked at runtime.

3 Principal Risks and Pitfalls

3.1 Runtime Contracts Are Absent

Preconditions, postconditions, refinements, loop invariants, decreases clauses, and unreachable proofs are not enforced during execution. A program may compile and pass ordinary tests while violating a declared contract. Documentation currently discloses this, but tooling and users can still overinterpret contract acceptance.

Priority: high. Runtime contract semantics should be implemented before Sol is presented as an application language with progressive verification.

3.2 No Application Boundary

The compiler does not define which declaration is an application entrypoint, which signatures are legal, how capabilities are injected, how arguments are represented, how results map to process status, or how panic and runtime diagnostics are surfaced.

Entrypoint semantics should be settled before a backend embeds accidental naming or ABI conventions.

3.3 Unsafe Host Error String

The host callback failure path returns an unconstrained C `const char *` that is consumed as a NUL-terminated string. A dangling or unterminated pointer can cause an out-of-bounds read. Successful host values receive substantial validation and cloning, but failed host messages do not have equivalent ownership or length guarantees.

Replace this boundary with an owned or length-delimited error value before expanding host integration.

3.4 Public C IR Is Not a Hostile-Input Format

The public C IR structure exposes mutable pointers and counts. Logical validation cannot prove that a non-null pointer actually addresses the advertised number of objects. Internally generated IR is strongly validated, but arbitrary foreign or corrupted C metadata cannot be made memory-safe by logical validation alone.

A future public serialized IR requires a separate checked decoder and owned representation.

3.5 Unbounded Compilation Resources

Interpreter execution has explicit limits, while compilation lacks comprehensive budgets for package bytes, file count, directory depth, tokens, syntax arenas, diagnostics, semantic graph size, and total allocation. Adversarial source can therefore consume excessive memory or native stack.

Package traversal is recursively stack-based and should gain a deterministic depth and resource budget.

3.6 Package Filesystem Races

Directory discovery inspects paths and later reopens them. Concurrent path replacement can change what is loaded after discovery, including replacing a regular file with a symlink. The formatter has stronger transactional checks than normal compilation.

Descriptor-based loading or post-open identity verification is required before packages become a security-sensitive build boundary.

3.7 Cubic Generic Call Analysis

Generic recursion analysis currently uses a dense declaration matrix and transitive closure. Its cost is approximately quadratic memory and cubic time in declaration count. Replace it with adjacency lists and strongly connected components before package scale increases substantially.

3.8 Repeated Whole-IR Validation

IR is validated during lowering, ownership analysis, interpreter startup, and repeatedly across authored tests. This can approach $O(\text{test count} * \text{package IR size})$ even though the IR is immutable.

A validated immutable IR handle or internal validation certificate would preserve safety without rerunning every full invariant pass for every test.

3.9 Syntax-Indexed Side Tables

HIR, types, effects, and contracts are broad parallel tables indexed by syntax IDs. Every new construct requires coordinated updates across many arenas and validators. This architecture also forces package-global invalidation and makes caching or serialization difficult.

Before incremental compilation or public semantic tooling, add an owned compilation session and consider a coherent typed semantic layer.

3.10 Duplicated Semantic Rules

Independent validation is valuable, but several semantic algorithms exist in parallel frontend and IR forms: match usefulness, finite inhabitation, contract purity, capability roots, callable effects, and ownership-related type recursion. Drift can either accept unsound metadata or reject valid frontend output as malformed IR.

Shared specifications, generated tables, or common constructor-domain utilities should define policy while independent validators continue to verify representation-specific facts.

3.11 Portability and Release Infrastructure

The build has MSVC branches, but production sources depend directly on POSIX APIs and filesystem behavior. Windows is not currently a realistic supported target. There is also no checked-in CI workflow, install/export target, release packaging, fuzz target, coverage gate, cross-platform matrix, or performance baseline.

Either introduce a platform layer or explicitly declare POSIX-only support for the current bootstrap.

3.12 Testing Gaps

The handwritten positive, negative, sanitizer, and malformed-metadata tests are strong. Missing categories include parser/package fuzzing, IR and runtime-value fuzzing, allocation-failure injection, generated properties, large-package stress, standard JSON Schema validation, and interpreter/backend differential execution.

4 Remaining Roadmap

The unfinished roadmap falls into five groups:

- Items 28-39: closures, richer traits, constants, numerics, arrays, lifetimes, resources, collections, and iterators.
- Items 40-44: manifests and dependencies, schemas, general effect polymorphism, cross-cutting semantic integration, and unsafe.
- Items 45-50: MIR, FFI, backend, runtime ABI, entrypoints/build/run, and executable contracts.

- Items 51-58: generated properties, SMT, formatter completion, public IR, language server, semantic patches, reports, and agent context.
- Items 59-61: concurrency, reflection/build transforms, and general handlers.

The current numeric order is not the shortest path to usable execution. Most language-breadth work in items 28-44 is not required to execute an application through the existing reference interpreter.

5 Recommended Endpoint

The shortest meaningful endpoint is a bounded `sol run` profile using the existing owning IR and reference interpreter, with explicit entrypoint and host-capability semantics, followed by executable runtime contracts.

This endpoint is not a production runtime. It is the smallest vertical slice that turns Sol from a language-test harness into an executable application model while exercising ownership, effects, authority, contracts, packages, diagnostics, and runtime behavior together.

6 Shortest End-to-End Path

6.1 Milestone 1: Shared Compilation Session and Hardening

Create one reusable compilation/session API instead of keeping orchestration private to the CLI and duplicating it in tests.

Exit criteria:

- Compile one file or package into validated immutable owning IR.
- Request check, inspection, effect, test, or run outputs through one phase owner.
- Centralize initialization, teardown, diagnostics, and phase ordering.
- Add package limits for bytes, files, depth, tokens, arenas, diagnostics, and allocation.
- Replace the unsafe host error pointer with an owned or length-delimited error.
- Replace cubic generic transitive closure with SCC analysis.
- Validate immutable IR once for repeated execution.

6.2 Milestone 2: Entrypoint and Application ABI

Split entrypoint semantics out of roadmap item 49 rather than waiting for a backend.

Define:

- An explicit entrypoint declaration or annotation.
- Visibility and uniqueness rules.
- Allowed generic, parameter, return, effect, and contract shapes.
- Process exit-status mapping.
- Panic and runtime-failure behavior.
- Capability parameter injection.
- Default execution limits.
- Deterministic duplicate or missing-entry diagnostics.

A bounded shape could resemble:

```
public entry function main(console: capability Console) -> Int64
effects { console.write<console> } {
  console.write("hello")
  return 0
}
```

Entrypoint identity should be retained explicitly in owning IR rather than inferred from a function-name scan.

6.3 Milestone 3: Minimal Trusted Host Profile

Split interpreter host support out of roadmap item 48.

Implement only what a representative application requires:

- Console output.

- A bounded process-argument representation.
- Optional deterministic configuration input.
- Explicit capability-root registry.
- Operation allowlist.
- Stable owned host failures.
- Default and configurable execution limits.

Filesystem, network, clock, randomness, and process mutation should remain absent until each receives an explicit authority and deterministic-test policy.

6.4 Milestone 4: `sol run`

Add:

```
sol run <file.sol|package-directory>
```

Required behavior:

- Compile through the shared session.
- Resolve exactly one entrypoint.
- Verify all required host capabilities before execution.
- Free frontend state before running owning IR.
- Execute under deterministic limits.
- Map return, panic, contract failure, and runtime failure to process status.
- Emit human diagnostics and a versioned JSON runtime envelope.

At this point Sol has meaningful end-to-end application execution.

6.5 Milestone 5: Executable Contracts

Split runtime contract checking out of roadmap item 50.

Implement:

- requires before callable body execution.
- old snapshots at callable entry.
- ensures after successful return.
- `Result` success/failure outcome clauses.
- Runtime refined-value construction and validation.
- Structured contract-failure diagnostics.
- Exact ownership cleanup when a contract fails.

Loop invariant and decreases checking may follow as a second increment if needed.

6.6 Milestone 6: Representative Application and Conformance

Add one multi-file executable fixture exercising imports, semantic IDs, records, enums, tuples, generics, traits, recursive matching, ownership, mutation, `Option/Result`, capability-injected console output, contracts, runtime failures, and cleanup.

The same fixture should pass through `sol fmt`, `sol check`, `sol test`, `sol effects`, `sol inspect`, and `sol run`. This becomes the acceptance test for the versioned executable-core profile.

7 Recommended Immediate Sequence

current item 27

- > shared compilation session and hardening
- > entrypoint/application ABI
- > minimal interpreter host profile
- > `sol run`
- > executable contracts/refinements
- > representative application and conformance suite

Items 28-47 can mostly be deferred for this endpoint.

8 Production Path

After interpreter-based application execution stabilizes the semantics, the shortest production route is likely WebAssembly-first:

```
entrypoint/application ABI
-> ownership-explicit CFG/MIR for the current subset
-> monomorphization and representation strategy
-> target-independent runtime ABI
-> WebAssembly backend
-> WebAssembly host adapter
-> sol build
-> interpreter/WebAssembly differential suite
```

The production roadmap must explicitly cover generic monomorphization or dictionaries, aggregate and text layout, symbols and linkage, Wasm imports/exports, runtime memory management, panic and cleanup policy, source maps, artifact metadata, reproducibility, release CI, fuzzing, security review, and performance gates.

9 Roadmap Restructure

Recommended splits:

- Item 40: application metadata; dependencies and locks; visibility and re-exports; sandboxed builds.
- Item 48: interpreter host profile; target-independent runtime ABI; backend adapters.
- Item 49: entrypoint semantics; interpreter `sol run`; backend `sol build`.
- Item 50: runtime contracts; normalized logical obligations; refinement-aware matching; cost/resource checks.
- Item 43: immediate integration acceptance criteria rather than one permanent mega-task.

Recommended new explicit tasks:

- Versioned executable-core conformance suite.
- Shared compilation/session API.
- Compiler resource budgets.
- Host capability registry.
- Representative executable application.
- Monomorphization and data-layout strategy.
- Interpreter/backend differential testing.
- Fuzzing, allocation-failure testing, clean-build CI, and release hardening.

10 Deferred Work

For the interpreter-based application endpoint, defer closures and richer traits, broad numeric and collection support, full package dependencies, general effect polymorphism, unsafe, C FFI, MIR/backend work, SMT, public IR, language-server functionality, semantic patches, concurrency, reflection, and general resumptive handlers.

Only a minimal package/application profile is required before `sol run`.

11 Conclusion

The frontend/interpreter core is substantially complete and internally coherent. The project should pause language breadth.

The shortest route to demonstrable end-to-end value is:

1. Harden and centralize compilation.
2. Define entrypoint and host semantics.
3. Add `sol run` over the existing interpreter.
4. Make contracts executable.
5. Prove the workflow with one representative application.

Only after these semantics are stable should they be embedded into MIR and a WebAssembly or native backend.